

循序渐进，学习开发 xv6-riscv

第 20 章 基于内存的文件系统

汪辰



目录

01

基于内存的简易文件系统

02

改进 Shell

03

添加更多的程序文件

A decorative graphic on the left side of the slide. It features two concentric circles with a light blue glow. A thin grey arc with dots at its ends is positioned around the circles. In the top right corner, there is a solid blue rectangle.

01

基于内存的简易文件系统

现有应用程序构建和组织管理方式

Makefile

```
_%: %.o $(ULIB) $U/user.ld
    $(LD) $(LDFLAGS) -T $U/user.ld -o $@ $< $(ULIB)
    $(OBJDUMP) -S $@ > $*.asm
    $(OBJDUMP) -t $@ | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$$/d' > $*.sym
    $U/gen_ccode.sh $@ $@.c
    mv $@.c $K/
```

Makefile

```
OBJS = \
    # .....
    $K/_init.o \
    $K/_sh.o \
    # .....
$K/kernel: $U/_init $U/_sh $(OBJS) $K/kernel.ld
    $(LD) $(LDFLAGS) -T $K/kernel.ld -o $K/kernel $(OBJS)
    $(OBJDUMP) -S $K/kernel > $K/kernel.asm
    $(OBJDUMP) -t $K/kernel | sed '1,/SYMBOL TABLE/d; s/ .
# .....
$K/_init.o: $K/_init.c
    $(CC) $(CFLAGS) -c -o $K/_init.o $K/_init.c
$K/_sh.o: $K/_sh.c
    $(CC) $(CFLAGS) -c -o $K/_sh.o $K/_sh.c
# .....
```

kernel/exec.c

```
// .....
extern uchar program__init[]; // The program
extern uchar program__sh[];   // The shell.
// .....
int
kexec(char *path, char **argv)
{
    // .....
    // Open the executable file.
    if (strncmp(path, "init", 5) == 0) {
        program = program__init;
    } else if (strncmp(path, "sh", 2) == 0) {
        program = program__sh;
    } else
        goto bad;
    // .....
```

现有应用程序构建和组织管理方式

Makefile

```
_%: %.o $(ULIB) $U/user.ld
$(LD) $(LDFLAGS) -T $U/user.ld -o $@ $< $(ULIB)
$(OBJDUMP) -S $@ > $*.asm
$(OBJDUMP) -t $@ | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$$/d' > $*.sym
$U/gen_ccode.sh $@ $@.c
mv $@.c $K/
```

Makefile

```
OBJS = \
# .....
$K/_init.o \
$K/_sh.o \
# .....
$K/kernel: $U/_init $U/_sh $(OBJS) $K/kernel.ld
$(LD) $(LDFLAGS) -T $K/kernel.ld -o $K/kernel $(OBJS)
$(OBJDUMP) -S $K/kernel > $K/kernel.asm
$(OBJDUMP) -t $K/kernel | sed '1,/SYMBOL TABLE/d' > $K/kernel.sym
# .....
$K/_init.o: $K/_init.c
$(CC) $(CFLAGS) -c -o $K/_init.o $K/_init.c
$K/_sh.o: $K/_sh.c
$(CC) $(CFLAGS) -c -o $K/_sh.o $K/_sh.c
# .....
```

重复操作

kernel/exec.c

```
// .....
extern uchar program__init[]; // The program
extern uchar program__sh[]; // The shell.
// .....
int
exec(char *path, char **argv)
{
// .....
// Open the executable file.
if (strcmp(path, "init", 5) == 0) {
    program = program__init;
} else if (strcmp(path, "sh", 2) == 0) {
    program = program__sh;
} else
    goto bad;
// .....
```

现有应用程序构建和组织管理方式

Makefile

```
_%: %.o $(ULIB) $U/user.ld
    $(LD) $(LDFLAGS) -T $U/user.ld -o $@ $< $(ULIB)
    $(OBJDUMP) -S $@ > $*.asm
    $(OBJDUMP) -t $@ | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$$/d' > $*.sym
    $U/gen_ccode.sh $@ $@.c
    mv $@.c $K/
```

Makefile

```
OBJS = \
# .....
$K/_init.o \
$K/_sh.o \
# .....
$K/kernel: $U/_init $U/_sh $(OBJS) $K/kernel.ld
    $(LD) $(LDFLAGS) -T $K/kernel.ld -o $K/kernel $(OBJS)
    $(OBJDUMP) -S $K/kernel > $K/kernel.asm
    $(OBJDUMP) -t $K/kernel | sed '1,/SYMBOL TABLE/d' > $K/kernel.sym
# .....
$K/_init.o: $K/_init.c
    $(CC) $(CFLAGS) -c -o $K/_init.o $K/_init.c
$K/_sh.o: $K/_sh.c
    $(CC) $(CFLAGS) -c -o $K/_sh.o $K/_sh.c
# .....
```

重复操作

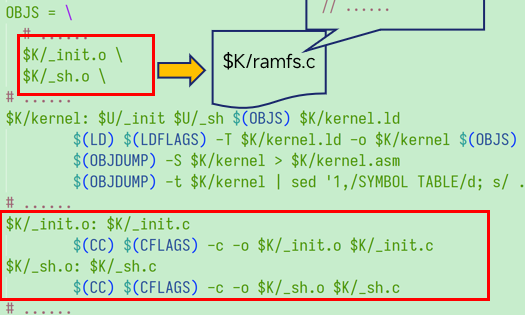
现有应用程序构建和组织管理方式

Makefile

```
_%: %.o $(ULIB) $U/user.ld
    $(LD) $(LDFLAGS) -T $U/user.ld -o $@ $< $(ULIB)
    $(OBJDUMP) -S $@ > $*.asm
    $(OBJDUMP) -t $@ | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$$/d' > $*.sym
    $U/gen_ccode.sh $@ $@.c
    mv $@.c $K/
```

Makefile

```
OBJS = \
# .....
$K/_init.o \
$K/_sh.o \
# .....
$K/kernel: $U/_init $U/_sh $(OBJS) $K/kernel.ld
    $(LD) $(LDFLAGS) -T $K/kernel.ld -o $K/kernel $(OBJS)
    $(OBJDUMP) -S $K/kernel > $K/kernel.asm
    $(OBJDUMP) -t $K/kernel | sed '1,/SYMBOL TABLE/d; s/ .
# .....
$K/_init.o: $K/_init.c
    $(CC) $(CFLAGS) -c -o $K/_init.o $K/_init.c
$K/_sh.o: $K/_sh.c
    $(CC) $(CFLAGS) -c -o $K/_sh.o $K/_sh.c
# .....
```



```
%: %.o $(ULIB) $U/user.ld
$(LD) $(LDFLAGS) -T $U/user.ld $(OBJS) $(LIBS) -o $@
$(OBJDUMP) -S $@ > $*.asm
$(OBJDUMP) -t $@ | sed '1,10000p' > $*.info
$U/gen_ccode.sh $@ $@.c
mv $@.c $K/
```

```

OBJS = \
# .....
$K/_init.o \
$K/_sh.o \
# .....
$K/kernel: $U/_init $U/_sh $(OBJS) $K/kernel.ld
$(LD) $(LDFLAGS) -T $K/kernel.ld -o $K/kernel $(OBJS)
$(OBJDUMP) -S $K/kernel > $K/kernel.asm
$(OBJDUMP) -t $K/kernel | sed '1,/SYMBOL TABLE/d; s/
# .....
$K/_init.o: $K/_init.c
$(CC) $(CFLAGS) -c -o $K/_init.o $K/_init.c
$K/_sh.o: $K/_sh.c
$(CC) $(CFLAGS) -c -o $K/_sh.o $K/_sh.c
# .....

```

```
#include "_init.c"
#include "_sh.c"
// .....
```

--	--

现有应用程序构建和组织管理方式

Makefile

```
_: %.o $(ULIB) $U/user.ld
$(LD) $(LDFLAGS) -T $U/user.ld -o $@ $< $(ULIB)
$(OBJDUMP) -S $@ > $*.asm
$(OBJDUMP) -t $@ | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$$/d' > $*.sym
$U/gen_ccode.sh $@ $@.c
mv $@.c $K/
```

kernel/exec.c

```
// .....
extern uchar program__init[]; // The program
extern uchar program__sh[];  // The shell.
// .....
int
exec(char *path, char **argv)
{
    // .....
    // Open the executable file.
    if (strncmp(path, "init", 5) == 0) {
        program = program__init;
    } else if (strncmp(path, "sh", 2) == 0) {
        program = program__sh;
    } else
        goto bad;
    // .....
}
```

重复操作

现有应用程序构建和组织管理方式

Makefile

kernel/ramfs.c

```
_%: %.o $(ULIB) $U/user.ld
```

```
$(LD) $(LDFLAGS) -T $U/user.ld -o $@ $< $(ULIB)
```

```
,/SYMBOL TABLE/d; s/ .* / /; /^$$/d' > $*.sym
```

```
#include "_init.c"
#include "_sh.c"
```

```
struct ram_file {
    char *name;
    uchar len_name;
    uchar *program;
};
```

```
static const struct ram_file ramfs[] = {
    {"init", 5, program__init},
    {"sh", 2, program__sh},
};
```

```
uchar * find_program(char *name)
```

```
{
    for (int i = 0; i < sizeof(ramfs) / sizeof(struct ram_file); i++) {
        if (strcmp(name, ramfs[i].name, ramfs[i].len_name) == 0) {
            return ramfs[i].program;
        }
    }
    return 0; // Not found
}
```

kernel/exec.c

```
// .....
extern uchar program__init[]; // The program
extern uchar program__sh[];   // The shell.
// .....
int
kexec(char *path, char **argv)
{
    // .....
    // Open the executable file.
    if (strcmp(path, "init", 5) == 0) {
        program = program__init;
    } else if (strcmp(path, "sh", 2) == 0) {
        program = program__sh;
    } else
        goto bad;
    // .....
```

现有应用程序构建和组织管理方式

Makefile

kernel/ramfs.c

```
_%: %.o $(ULIB) $U/user.ld
      $(LD) $(LDFLAGS) -T $U/user.ld -o $@ $< $(ULIB)
```

```
#include "_init.c"
#include "_sh.c"

struct ram_file {
    char *name;
    uchar len_name;
    uchar *program;
};

static const struct ram_file ramfs[] = {
    {"init", 5, program__init},
    {"sh", 2, program__sh},
};

uchar * find_program(char *name)
{
    for (int i = 0; i < sizeof(ramfs) / sizeof
        if (strcmp(name, ramfs[i].name, ramfs[
            return ramfs[i].program;
        }
    }
    return 0; // Not found
}
```

```
diff --git a/kernel/exec.c b/kernel/exec.c
index bd09dc0..b705501 100644
--- a/kernel/exec.c
+++ b/kernel/exec.c
@@ -20,8 +20,7 @@ int flags2perm(int flags)
     return perm;
 }

-extern uchar program__init[]; // The program to run at init
-extern uchar program__sh[]; // The shell.
+extern uchar *find_program(char *path);

//
// the implementation of the exec() system call
@@ -37,14 +36,9 @@ kexec(char *path, char **argv)
    pagetable_t pagetable = 0, oldpagetable;
    struct proc *p = myproc();

-    uchar *program = 0;
-
-    // Open the executable file.
-    if (strcmp(path, "init", 5) == 0) {
-        program = program__init;
-    } else if (strcmp(path, "sh", 2) == 0) {
-        program = program__sh;
-    } else
-        goto bad;
+    uchar *program = find_program(path);
+    if (program == 0)
+        goto bad;
```

```
.* / /; / ^$/d' > $*.sym
```

kernel/exec.c

```
// .....
extern uchar program__init[]; // The program
extern uchar program__sh[]; // The shell.
// .....

int
kexec(char *path, char **argv)
{
    // .....
    // Open the executable file.
    if (strcmp(path, "init", 5) == 0) {
        program = program__init;
    } else if (strcmp(path, "sh", 2) == 0) {
        program = program__sh;
    } else
        goto bad;
    // .....
```

现有应用程序构建和组织管理方式

文件系统

init
sh
.....

kernel/ramfs.c

```
#include "_init.c"
#include "_sh.c"

struct ram_file {
    char *name;
    uchar len_name;
    uchar *program;
};

static const struct ram_file ramfs[] = {
    {"init", 5, program__init},
    {"sh", 2, program__sh},
};

uchar * find_program(char *name)
{
    for (int i = 0; i < sizeof(ramfs) / sizeof
        if (strcmp(name, ramfs[i].name, ramfs[
            return ramfs[i].program;
        }
    }
    return 0; // Not found
}
```

B) \$U/user.ld

\$(LDFLAGS) -T \$U/user.ld -o \$@ \$< \$(ULIB)

```
diff --git a/kernel/exec.c b/kernel/exec.c
index bd09dc0..b705501 100644
--- a/kernel/exec.c
+++ b/kernel/exec.c
@@ -20,8 +20,7 @@ int flags2perm(int flags)
    return perm;
}
```

```
-extern uchar program__init[]; // The program to run at init
-extern uchar program__sh[]; // The shell.
+extern uchar *find_program(char *path);
```

```
//
// the implementation of the exec() system call
@@ -37,14 +36,9 @@ kexec(char *path, char **argv)
    pagetable_t pagetable = 0, oldpagetable;
    struct proc *p = myproc();
```

```
- uchar *program = 0;
-
- // Open the executable file.
- if (strcmp(path, "init", 5) == 0) {
-     program = program__init;
- } else if (strcmp(path, "sh", 2) == 0) {
-     program = program__sh;
- } else
-     goto bad;
+ uchar *program = find_program(path);
+ if (program == 0)
+     goto bad;
```

.* / /; / ^\$/d' > \$*.sym

kernel/exec.c

```
// .....
extern uchar program__init[]; // The program
extern uchar program__sh[]; // The shell.
```

```
// .....
int
kexec(char *path, char **argv)
{
```

```
// .....
// Open the executable file.
if (strcmp(path, "init", 5) == 0) {
    program = program__init;
} else if (strcmp(path, "sh", 2) == 0) {
    program = program__sh;
} else
    goto bad;
// .....
```



02

改进 Shell

Shell 改进

```
int
main(void)
{
    static char buf[100];

    // Read and run input commands.
    while(getcmd(buf, sizeof(buf)) >= 0){
        char *cmd = buf;
        while (*cmd == ' ' || *cmd == '\t')
            cmd++;
        if (*cmd == '\n') // is a blank command
            continue;
        if(fork1() == 0)
            runcmd(parsecmd(cmd));
        wait(0);
    }
    exit(0);
}
```

```
int
fork1(void)
{
    int pid;

    pid = fork();
    if(pid == -1)
        panic("fork");
    return pid;
}
```

Shell 改进

```
// Parsed command representation  
#define EXEC 1  
#define REDIR 2  
#define PIPE 3  
#define LIST 4  
#define BACK 5
```

Shell 改进

```
// Parsed command representat  
#define EXEC 1  
#define REDIR 2  
#define PIPE 3  
#define LIST 4  
#define BACK 5
```

- REDIR: 重定向
- PIPE: 管道

现阶段尚未支持

Shell 改进

```
// Parsed command representation
```

```
#define EXEC 1
```

```
#define REDIR 2
```

```
#define PIPE 3
```

```
#define LIST 4
```

```
#define BACK 5
```

- EXEC:普通命令执行

例子:

```
$ echo hello world!  
hello world!
```

```
case EXEC:
```

```
    ecmd = (struct execcmd*)cmd;
```

```
    if(ecmd->argv[0] == 0)
```

```
        exit(1);
```

```
    exec(ecmd->argv[0], ecmd->argv);
```

```
    fprintf(2, "exec %s failed\n", ecmd->argv[0]);
```

```
    break;
```

Shell 改进

```
// Parsed command representation  
#define EXEC 1  
#define REDIR 2  
#define PIPE 3  
#define LIST 4  
#define BACK 5
```

- LIST: 多个命令以分号“;”分隔顺序执行

例子:

```
$ echo 1; echo 2; echo 3; echo 4
```

```
1  
2  
3  
4
```



```
case LIST:
```

```
    lcmd = (struct listcmd*)cmd;
```

```
    if(fork1() == 0)
```

```
        runcmd(lcmd->left);
```

```
    wait(0);
```

```
    runcmd(lcmd->right);
```

```
    break;
```

Shell 改进

```
// Parsed command representation
#define EXEC 1
#define REDIR 2
#define PIPE 3
#define LIST 4
#define BACK 5
```

- BACK: 在后台执行命令

例子:

```
$ echo hello world! &
hello world!
```

```
case BACK:
```

```
    bcmd = (struct backcmd*)cmd;
    if(fork1() == 0)
        runcmd(bcmd->cmd);
    break;
```



03

添加更多程序文件



添加更多程序文件

```
diff --git a/Makefile b/Makefile
index 6ff3467..b69a3cf 100644
--- a/Makefile
+++ b/Makefile
@@ -113,8 +113,11 @@ $U/usys.o : $U/usys.S
 .PRECIOUS: %.o

UPROGS=\
+   $U/_echo\
+   $U/_init\
-   $U/_sh \
+   $U/_kill\
+   $U/_sh\
+   $U/_zombie\
```

```
diff --git a/kernel/ramfs.c b/kernel/ramfs.c
index ec4405f..26b0cea 100644
--- a/kernel/ramfs.c
+++ b/kernel/ramfs.c
@@ -4,6 +4,9 @@

#include "_init.c"
#include "_sh.c"
+include "_kill.c"
+include "_echo.c"
+include "_zombie.c"

struct ram_file {
    char *name;
@@ -14,6 +17,9 @@ struct ram_file {
 static const struct ram_file ramfs[] = {
     {"init", 5, program__init},
     {"sh", 2, program__sh},
+    {"kill", 4, program__kill},
+    {"echo", 4, program__echo},
+    {"zombie", 6, program__zombie},
 };
```

echo

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
    int i;

    for(i = 1; i < argc; i++){
        write(1, argv[i], strlen(argv[i]));
        if(i + 1 < argc){
            write(1, " ", 1);
        } else {
            write(1, "\n", 1);
        }
    }
    exit(0);
}
```

```
$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m
128M -smp 3 -nographic

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ echo hello world!
hello world!
$
```

kill

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char **argv)
{
    int i;

    if(argc < 2){
        fprintf(2, "usage: kill pid...\n");
        exit(1);
    }
    for(i=1; i<argc; i++){
        kill(atoi(argv[i]));
    }
    exit(0);
}
```

```
$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m
128M -smp 3 -nographic

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$
1 x sleep  init
2 1 sleep  sh
```

Ctrl + P

kill

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char **argv)
{
    int i;

    if(argc < 2){
        fprintf(2, "usage: kill pid...\n");
        exit(1);
    }
    for(i=1; i<argc; i++)
        kill(atoi(argv[i]));
    exit(0);
}
```

```
$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m
128M -smp 3 -nographic

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$
1 x sleep  init
2 1 sleep  sh
$ kill 2
init: starting sh
$
```

Ctrl + P

kill

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char **argv)
{
    int i;

    if(argc < 2){
        fprintf(2, "usage: kill pid...\n");
        exit(1);
    }
    for(i=1; i<argc; i++)
        kill(atoi(argv[i]));
    exit(0);
}
```

```
$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m
128M -smp 3 -nographic
```

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh

```
$
1 x sleep  init
2 1 sleep  sh
```

Ctrl + P

```
$ kill 2
init: starting sh
$
1 x sleep  init
4 1 sleep  sh
```

Ctrl + P

kill

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char **argv)
{
    int i;

    if(argc < 2){
        fprintf(2, "usage: kill pid...\n");
        exit(1);
    }
    for(i=1; i<argc; i++){
        kill(atoi(argv[i]));
    }
    exit(0);
}
```

```
$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m
128M -smp 3 -nographic
```

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh

```
$
1 x sleep init
2 1 sleep sh
```

Ctrl + P

```
$ kill 2
init: starting sh
$
1 x sleep init
4 1 sleep sh
```

Ctrl + P

```
int main(void)
{
    static char buf[100];

    // Read and run input commands.
    while(getcmd(buf, sizeof(buf)) >= 0){
        char *cmd = buf;
        while (*cmd == ' ' || *cmd == '\t')
            cmd++;
        if (*cmd == '\n') // is a blank command
            continue;
        if(fork1() == 0)
            runcmd(parsecmd(cmd));
        wait(0);
    }
    exit(0);
}
```

zombie

```
int main(void)
{
    printf("zombie: starting\n");
    int pid = fork();
    if(pid < 0) {
        printf("zombie: fork failed\n");
        exit(1);
    }

    // Parent exits first, leaving child as a zombie.
    // The child will be reparented to init.
    if (pid > 0) {
        pause(50);
        printf("zombie: parent exiting\n");
    } else {
        pause(100);
        printf("zombie: child exiting\n");
    }
    exit(0);
}
```

```
.....
xv6 kernel is booting
```

```
hart 1 starting
hart 2 starting
init: starting sh
$
1 x sleep  init
2 1 sleep  sh
```

Ctrl + P

zombie

```
int main(void)
{
    printf("zombie: starting\n");
    int pid = fork();
    if(pid < 0) {
        printf("zombie: fork failed\n");
        exit(1);
    }

    // Parent exits first, leaving child as a zombie.
    // The child will be reparented to init.
    if (pid > 0) {
        pause(50);
        printf("zombie: parent exiting\n");
    } else {
        pause(100);
        printf("zombie: child exiting\n");
    }
    exit(0);
}
```

```
.....
xv6 kernel is booting
```

```
hart 1 starting
hart 2 starting
init: starting sh
$
1 x sleep  init
2 1 sleep  sh
```

Ctrl + P

```
$ zombie
zombie: starting
```

zombie

```
int main(void)
{
    printf("zombie: starting\n");
    int pid = fork();
    if(pid < 0) {
        printf("zombie: fork failed\n");
        exit(1);
    }

    // Parent exits first, leaving child as a zombie.
    // The child will be reparented to init.
    if (pid > 0) {
        pause(50);
        printf("zombie: parent exiting\n");
    } else {
        pause(100);
        printf("zombie: child exiting\n");
    }
    exit(0);
}
```

.....
xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh

\$
1 x sleep init
2 1 sleep sh

Ctrl + P

\$ zombie
zombie: starting

1 x sleep init
2 1 sleep sh
3 2 sleep zombie
4 3 sleep zombie

Ctrl + P

zombie

```
int main(void)
{
    printf("zombie: starting\n");
    int pid = fork();
    if(pid < 0) {
        printf("zombie: fork failed\n");
        exit(1);
    }

    // Parent exits first, leaving child as a zombie.
    // The child will be reparented to init.
    if (pid > 0) {
        pause(50);
        printf("zombie: parent exiting\n");
    } else {
        pause(100);
        printf("zombie: child exiting\n");
    }
    exit(0);
}
```

.....
xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh

\$
1 x sleep init
2 1 sleep sh

Ctrl + P

\$ zombie
zombie: starting

1 x sleep init
2 1 sleep sh
3 2 sleep zombie
4 3 sleep zombie
zombie: parent exiting

Ctrl + P

\$
1 x sleep init
2 1 sleep sh
4 1 sleep zombie

Ctrl + P

zombie

```
int main(void)
{
    printf("zombie: starting\n");
    int pid = fork();
    if(pid < 0) {
        printf("zombie: fork failed\n");
        exit(1);
    }

    // Parent exits first, leaving child as a zombie.
    // The child will be reparented to init.
    if (pid > 0) {
        pause(50);
        printf("zombie: parent exiting\n");
    } else {
        pause(100);
        printf("zombie: child exiting\n");
    }
    exit(0);
}
```

.....
xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh

\$
1 x sleep init
2 1 sleep sh

Ctrl + P

\$ zombie
zombie: starting

1 x sleep init
2 1 sleep sh
3 2 sleep zombie
4 3 sleep zombie
zombie: parent exiting

Ctrl + P

\$
1 x sleep init
2 1 sleep sh
4 1 sleep zombie
zombie: child exiting

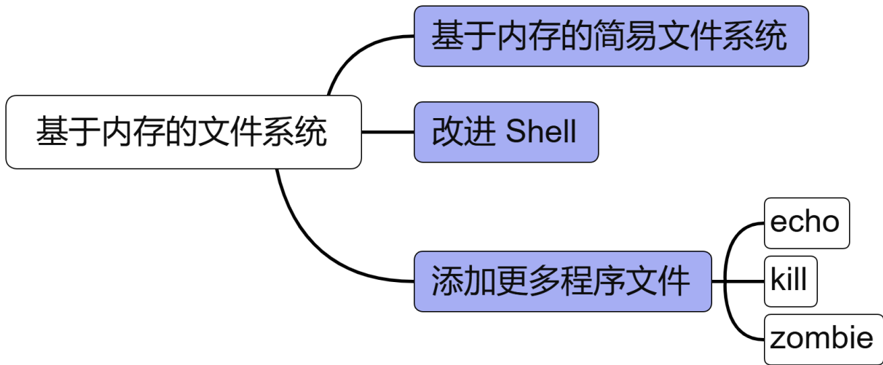
Ctrl + P

\$
1 x sleep init
2 1 sleep sh

Ctrl + P



本章总结



谢谢

